GUÍA DE IMPLEMENTACIÓN PRÁCTICA

Spring Cloud Config Server con Tres Microservicios

Caso Práctico: Sazón Express — Centralización de Configuraciones y Actualización en Caliente

Curso: Arquitectura de Software / Microservicios	**Tecnologías:** Java 17, Spring Boot 4.0.9, Spring Cloud 2025.1.4
Entorno: Windows / Maven 3.9 / Git	**Ecosistema:** 4 Módulos + Git config-repo + Actuator
Integrantes: Equipo de Proyecto Sazón Express	**Modalidad:** Exposición Académica Universitaria

1. Objetivo de la Guía

El propósito de esta guía es documentar de manera directa, reproducible y paso a paso la implementación de una arquitectura de microservicios basada en **Spring Cloud Config Server** para el sistema gastronómico **Sazón Express**. El docente y los estudiantes podrán levantar el sistema desde cero en cualquier computadora, demostrando cómo tres microservicios independientes (menu-service, pedido-service y promocion-service) obtienen sus políticas de negocio desde un repositorio **Git** y aplican actualizaciones dinámicas en tiempo de ejecución mediante @RefreshScope sin requerir bases de datos ni redespliegues.

2. Herramientas y Requisitos del Entorno

- **Java Development Kit (JDK 17 LTS o 21 LTS):** Plataforma base para compilación y ejecución de Spring Boot 4.x.
- **Apache Maven 3.9+ (o Maven Wrapper mvnw):** Gestión de dependencias y empaquetado JAR ejecutable.
- **Git 2.x:** Sistema de control de versiones que almacena las propiedades externas en config-repo.
- **PowerShell 5.1+ / 7.x (Windows):** Ejecución de los scripts automatizados de diagnóstico y pruebas.
- **Navegador web o cURL / Postman / Invoke-RestMethod:** Para consumir los endpoints REST y Actuator.

3. Creación de Proyectos con Spring Initializr

Cada uno de los cuatro módulos fue generado utilizando el servicio oficial **start.spring.io** con los siguientes parámetros base: Maven Project, Java 17, Spring Boot 4.0.9 y empaquetado Jar.

Módulo	Artifact ID	Puerto	Dependencias Initializr
Config Server	config-server	8888	Config Server, Test
Menú Service	menu-service	8081	Web (WebMVC), Config Client, Actuator
Pedido Service	pedido-service	8082	Web (WebMVC), Config Client, Actuator
Promoción Service	promocion-service	8083	Web (WebMVC), Config Client, Actuator

Capturas Reales de Configuración en Spring Initializr:

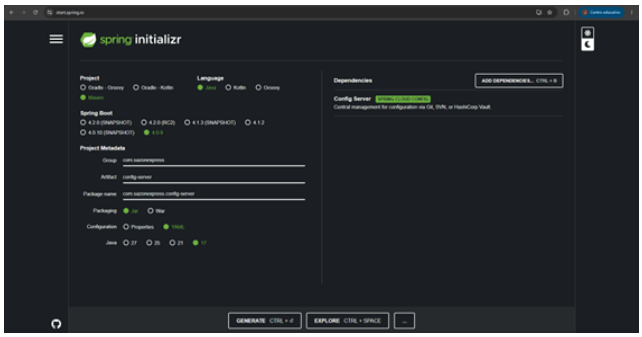


Figura 1: Creación de config-server (Config Server)



Figura 2: Creación de menu-service (Web, Client, Actuator)

4. Gestión de Dependencias (BOM Spring Cloud)

Para garantizar la compatibilidad entre Spring Boot 4.0.9 y Spring Cloud, se utiliza el BOM (*Bill of Materials*) oficial **spring-cloud-dependencies:2025.1.4** en todos los pom.xml:

```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>2025.1.4</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

5. Estructura General del Proyecto

El proyecto organiza las responsabilidades en 4 módulos independientes, 1 repositorio Git exclusivo de configuración y un conjunto de scripts PowerShell de automatización:

```
sazon-express/
■■■ pom.xml # Reactor agregador Maven raíz
■■■ config-server/ # Servidor Central (:8888)
■■■ menu-service/ # Microservicio de Menú (:8081)
■■■ pedido-service/ # Microservicio de Pedidos (:8082)
■■■ promocion-service/ # Microservicio de Promociones (:8083)
■■■ config-repo/ # Repositorio Git independiente (.git/)
■ ■■■ application.yml # Metadatos y Actuator común
■ ■■■ menu-service.yml # Carta y plato destacado
■ ■■■ pedido-service.yml # Límite max-platos y tiempos
■ ■■■ promocion-service.yml # Descuentos porcentuales
■■■ scripts/ # Scripts PowerShell (.ps1) de automatización
■■■ docs/ # PDF, PPTX y evidencias de ejecución
```

6. Configuración de Spring Cloud Config Server

Clase Principal: En ConfigServerApplication.java se incluye la anotación @EnableConfigServer.

Archivo application.yaml: Se enlaza el repositorio Git mediante protocolo file:/// o variable de entorno:

```
server:
port: 8888
spring:
application:
name: config-server
cloud:
config:
server:
git:
uri: ${CONFIG_REPO_URI:file:///${user.dir}/../config-repo}
default-label: main
clone-on-start: true
```

7. Creación del Repositorio Git (config-repo)

El repositorio config-repo es un repositorio Git auténtico e independiente. Para inicializarlo:

```
cd sazon-express/config-repo
git init
git config user.name "Sazon Express Team"
git config user.email "arquitectura@sazonexpress.com"
git add .
git commit -m "feat(config): configuracion inicial centralizada"
git branch -M main
```

8. Configuración de los Tres Clientes (Bootstrap)

En Spring Boot 4.x, los clientes conectan al Config Server mediante la propiedad spring.config.import en sus respectivos application.yaml locales:

```
spring:
application:
name: menu-service # (o pedido-service, promocion-service)
config:
import: "optional:configserver:${CONFIG_SERVER_URL:http://localhost:8888}"
management:
endpoints:
web:
exposure:
include: "health,info,refresh"
```

9. Programación de Endpoints y Uso de @RefreshScope

Para que los componentes recarguen sus valores en caliente cuando cambia el YAML en Git, se anotan con @RefreshScope junto con @ConfigurationProperties o @Service:

```
// MenuProperties.java
@Component
@RefreshScope
@ConfigurationProperties(prefix = "restaurante.menu")
public class MenuProperties {
private String nombre = "Carta Sazón Express";
private String platoDestacado = "Lomo saltado";
private String mensaje = "Disfruta nuestros sabores";
private boolean mostrarDisponibles = true;
// Getters y Setters...
}
```

10. Comandos de Compilación y Ejecución

El proyecto puede compilarse integralmente mediante el POM agregador raíz y ejecutarse de dos formas:

Opción A — Mediante Scripts Automatizados PowerShell (Recomendado):

```
cd sazon-express
.\scripts\verificar-entorno.ps1
.\scripts\iniciar-config-server.ps1
.\scripts\iniciar-microservicios.ps1
.\scripts\probar-endpoints.ps1
```

Opción B — Manualmente con Maven o Java JAR:

```
# 1. Compilar todo el sistema
mvn clean package -DskipTests

# 2. Iniciar Config Server (:8888)
java
-Dspring.cloud.config.server.git.uri=file:///D:/Proyects/microservicios/sazon-express/config-repo
-jar config-server/target/config-server-0.0.1-SNAPSHOT.jar

# 3. Iniciar clientes en consolas separadas (:8081, :8082, :8083)
java -jar menu-service/target/menu-service-0.0.1-SNAPSHOT.jar
java -jar pedido-service/target/pedido-service-0.0.1-SNAPSHOT.jar
java -jar promocion-service/target/promocion-service-0.0.1-SNAPSHOT.jar
```

11. Pruebas de Funcionamiento y Verificación

El proyecto incluye **23 pruebas automatizadas** ejecutadas con éxito con JUnit 5 y MockMvc. Para ejecutarlas todas:

```
mvn clean test
```

12. Demostración de Modificación Centralizada Dinámica

A continuación se presenta el flujo exacto ejecutado en la demostración universitaria:

- **Paso 1: Consultar estado actual:** GET http://localhost:8081/api/menu/destacado -> Retorna: 'Lomo saltado'.
- **Paso 2: Modificar propiedad en Git:** Editar config-repo/menu-service.yml: restaurante.menu.plato-destacado: 'Ceviche clásico'.
- **Paso 3: Confirmar commit en Git:** git add . && git commit -m 'feat: cambiar plato destacado a Ceviche clasico'.
- **Paso 4: Verificar en Config Server:** GET http://localhost:8888/menu-service/default -> Muestra el nuevo commit hash.
- **Paso 5: Invocar Refresh en Cliente:** POST http://localhost:8081/actuator/refresh con cabecera Content-Type: application/json.
- **Paso 6: Validar nuevo comportamiento:** GET http://localhost:8081/api/menu/destacado -> Retorna: 'Ceviche clásico' sin reiniciar Java!

13. Problemas Frecuentes y Soluciones

- **Error 415 al llamar /actuator/refresh:**
  Solución: Al enviar la petición POST, se debe especificar la cabecera -ContentType 'application/json' y el cuerpo -Body '{}'.
- **Connection refused en :8888 al arrancar clientes:**
  Solución: Config Server debe haber iniciado completamente antes de que los clientes intenten inicializarse. Utilice el script iniciar-config-server.ps1 que espera la disponibilidad del puerto.
- **Address already in use (Puertos 8081-8083 u 8888 ocupados):**
  Solución: Ejecute el script .\scripts\detener-servicios.ps1 para liberar los procesos Java en escucha automáticamente.
- **JGit remote null warning en logs:**
  Solución: Es una advertencia normal cuando el repositorio Git es estrictamente local y no cuenta con un remote 'origin' configurado. No afecta el funcionamiento.

14. Conclusiones

- Spring Cloud Config Server centraliza eficientemente la gestión de microservicios, eliminando archivos dispersos y desalineaciones entre entornos.
- El uso de Git como backend garantiza trazabilidad, inmutabilidad de versiones y alineación con las mejores prácticas de la industria (GitOps / Configuration-as-Code).
- La integración con Spring Boot Actuator y @RefreshScope permite actualizar políticas operativas y comerciales en caliente con cero tiempo de inactividad (Zero Downtime).
- El diseño en memoria implementado en Sazón Express permite una demostración limpia, pedagógica y 100% reproducible en cualquier máquina académica sin fricción de bases de datos.